

## Báo cáo rà soát test code

<!-- framework: Jest | run\_id: run\_20260620\_030234\_642738 -->

---

Framework: Jest | Run ID: run\_20260620\_030234\_642738

---

### Tóm tắt kỹ thuật

- Framework: Jest (CommonJS)
- Kết quả chạy: Tất cả 12 test được sinh (TC-001-014, trừ TC-003/004) PASS; coverage 90 % → đạt quality gate.
- Vấn đề lớn nhất: Các test cho roundMoney (TC-003, TC-004) không tồn tại → thiếu kiểm thử chức năng quan trọng, gây lỗi hổng trong bộ kiểm thử.
- Tài liệu kiểm thử được xem xét: generated.test.js (test path) và orderPricing.js (source\_under\_test.js).

### Lỗi nghiêm trọng

- Không phát hiện.

### Test còn thiếu

- TC-003 & TC-004: Hai kịch bản “roundMoney rounds up/down correctly” được liệt kê trong Test Plan nhưng không có trong mã test sinh ra.
- Bằng chứng: Test Plan có id: "TC-003" và id: "TC-004"; trong generated.test.js không có test nào có tiêu đề chứa “TC-003” hoặc “TC-004”.
- Ảnh hưởng: roundMoney là hàm nội bộ nhưng được yêu cầu trong yêu cầu (định dạng làm tròn 2 chữ số). Việc thiếu test làm giảm độ tin cậy và không đáp ứng đầy đủ Test Plan.
- Cách sửa: Thêm hai test tương tự:

```
test('TC-003 roundMoney rounds up correctly', () => {  
  expect(target.roundMoney(12.345)).toBeCloseTo(12.35);  
});
```

```
test('TC-004 roundMoney rounds down correctly', () => {  
  expect(target.roundMoney(7.344)).toBeCloseTo(7.34);  
});
```

(Nếu roundMoney không được export, cần export nó trong orderPricing.js hoặc kiểm thử thông qua hàm công khai có sử dụng roundMoney.)

- TC-014 (phần “shippingFee behavior not validated”): Test hiện tại chỉ kiểm tra kiểu trả về và không xác thực logic tính phí vận chuyển, trong khi yêu cầu giả định shippingFee tồn tại và ảnh hưởng tới finalTotal.
- Bằng chứng: Test TC-014 chỉ có:

```
expect(typeof result).toBe('number');  
expect(() => target.finalTotal(order)).not.toThrow();
```

- Ảnh hưởng: Không phát hiện lỗi tiềm ẩn trong tính toán shippingFee (ví dụ: trọng lượng >20kg, destination không hợp lệ).
- Cách sửa: Thêm các trường hợp kiểm tra giá trị finalTotal dựa trên các kịch bản shippingFee đã biết, ví dụ:

```
test('TC-014 finalTotal integrates shippingFee correctly for international fragile shipment', () => {
  const order = { storeCredit: 0, subtotal: 100, customerTier: 'standard',
    weightKg: 25, destination: 'international', fragile: true };
  const discount = target.calculateDiscount(order.subtotal, order.customerTier);
  const shipping = target.shippingFee(order.weightKg, order.destination, order.fragile);
  const expected = target.roundMoney(Math.max(order.subtotal - discount + shipping, 0));
  expect(target.finalTotal(order)).toBeCloseTo(expected);
});
```

#### Assertion yếu

- TC-014: Assertion chỉ kiểm tra kiểu dữ liệu và không kiểm chứng giá trị tính toán, dẫn tới khả năng test “vàng” (luôn pass).
- Bằng chứng: `expect(typeof result).toBe('number');` và `expect(() => target.finalTotal(order)).not.toThrow();` không so sánh với giá trị mong đợi.
- Ảnh hưởng: Không phát hiện sai lệch trong công thức `finalTotal`.
- Cách sửa: Thay thế bằng so sánh giá trị thực tế như trong đề xuất ở mục “Test còn thiếu”.

#### Rủi ro maintainability

- Dependency vào hàm nội bộ `roundMoney` không được export: Test TC-011 và TC-012 tự định nghĩa lại `roundMoney` trong file test, gây rủi ro nếu logic thay đổi trong source nhưng không được cập nhật trong test.
- Bằng chứng: Đoạn function `roundMoney(value) { return Math.round(value * 100) / 100; }` trong `generated.test.js`.
- Ảnh hưởng: Khi `roundMoney` được sửa (ví dụ: thay đổi cách làm tròn), các test hiện tại sẽ vẫn sử dụng phiên bản cũ, dẫn tới false-positive.
- Cách sửa: Export `roundMoney` từ `orderPricing.js` và sử dụng `target.roundMoney` trong test, hoặc tính toán kỳ vọng bằng cách gọi trực tiếp hàm đã export.

#### Gợi ý sửa ngay

1. Thêm test cho TC-003 và TC-004 (xem mục “Test còn thiếu”).
2. Cập nhật TC-014 để so sánh giá trị `finalTotal` với kết quả tính toán dựa trên `shippingFee`.
3. Export `roundMoney` từ source và thay thế hàm tự định nghĩa trong test bằng `target.roundMoney`.
4. Kiểm tra lại coverage sau khi bổ sung các test mới để duy trì  $\geq 90\%$  và tránh “dead code”.

---

\*Báo cáo này dựa trên bằng chứng thực tế từ mã nguồn, kế hoạch test và log thực thi.\*